

Executive Summary

This experiment implements a multi-step “safety” pipeline that transforms each input prompt into a **trust score (0–1)** by extracting hidden-state features from a language model and computing a series of statistics. The main stages are: (1) **Model encoding** (tokenize and run the prompt through the model to get hidden states), (2) **Variance analysis** (compute variances in mid and final layers), (3) **Embedding extraction** (collapse the final-layer hidden states into a single vector and normalize it), (4) **Anchor updates** (compute “benign” and “malice” prototype vectors), (5) **Distance/outlier detection** (measure how far the prompt’s embedding is from known benign embeddings), (6) **Cosine similarity & margin** (compare the embedding to benign vs. malicious anchors and compute the “margin”), (7) **Penalty calculation** (derive intent and manifold penalties from the margin and distance), (8) **Z-score “Interlock” check** (compute how anomalous the margin is relative to a benign buffer of past margins), (9) **Trust score computation** (combine all signals via a logistic function), and (10) **Buffer updates** (update benign/malicious buffers based on purity and outlier conditions). An auxiliary **Convergence Watchdog** monitors drift of the benign margin distribution. The pipeline is summarized below and then detailed stage by stage with toy examples.

flowchart TD

```
A[Prompt Text] --> B(Tokenizer & Model Forward)
B --> C(Variance & Contrast Computation)
C --> D(Embedding Extraction & Normalization)
D --> E(Anchor Update: \nBenign & Malicious Vectors)
E --> F(Distance to Benign Manifold (NN Search))
F --> G(Cosine Simes to Anchors)
G --> H(Margin = cos_exploit - cos_research)
H --> I(Intent Penalty = max(0,(margin-θ)*K))
H --> J(Manifold Penalty = dist*PENALTY_MULT*factor)
H --> K(Z-Score from Benign Buffer)
K --> L(Interlock Penalty if Z > threshold)
I & J & L --> M(Base Signal & Final Signal)
M --> N(Trust Score = sigmoid(final_signal))
N --> O{Purity Gate: conditions}
O -->|Pass| P(Update benign buffer)
P --> Q(Update θ_exploit percentile)
O --> R(Update malicious buffer if outlier/blocked)
N --> S(Output trust score)
```

High-level Inputs/Outputs: The input is a raw text prompt. The outputs at each stage are numerical features (hidden states, variances, embeddings, distances, similarities, penalties) culminating in a scalar *trust_score* (higher means “safer/benign”). Below, each transformation is described with its purpose, formulas, and a small worked example.

Stage 1: Tokenization & Hidden-State Extraction

- **Input:** Raw text prompt (e.g. "Explain DNS hijacking.").
- **Operation:** The prompt is tokenized via a pretrained tokenizer and passed through the language model (`model(**inputs)`), yielding hidden states for each layer. In code, `h_mid = outputs.hidden_states[MID_LAYER]` and `h_final = outputs.hidden_states[-1]` . We do **not** detail tokenization math (standard). Rather, note that if `h_mid` has shape `(1, N, d)` and `h_final` has shape `(1, N, d)` (batch=1, `N` tokens, hidden-dimension `d`), then each is a matrix of floats.
- **Output:** Two matrices of activations: one from the middle layer (`h_mid`) and one from the final layer (`h_final`). These represent the prompt's embedding across tokens at those layers. For example, a toy hidden state (with 2 tokens, dimension 2) might be:

h_mid (layer 12)	Token1	Token2
Feature1	1.0	3.0
Feature2	2.0	4.0
h_final (last layer)	Token1	Token2
Feature1	0.5	0.9
Feature2	0.1	0.7

(These numbers are illustrative; actual outputs come from the model.)

- **Purpose:** To obtain dense vector representations of the prompt. These matrices become the basis for all subsequent statistical features.

Stage 2: Variance Computation

- **Input:** The matrices `h_mid` and `h_final` from Stage 1.
- **Operation:** Compute the variances of each matrix's entries and their ratio and difference. Concretely:
 - `var_mid = Var( $h_{mid}$ )` (variance of all elements of `h_mid`),
 - `var_final = Var( $h_{final}$ )`.
- Then compute the **variance ratio**:

$$\text{var_ratio} = \frac{\text{var_final}}{\text{var_mid} + \epsilon},$$

where a tiny $\epsilon = 10^{-9}$ avoids division by zero.

- And the **contrast delta**:

$$\text{contrast_delta} = |\text{var_mid} - \text{var_final}|.$$

These capture whether the final-layer activations “collapse” (much lower variance) relative to the mid-layer.

- **Output:** Scalars `var_mid`, `var_final`, `var_ratio`, and `contrast_delta`.
- **Formula Example:** For our toy data above,
 - Compute mean of all elements: $\mu_{\text{mid}} = (1 + 2 + 3 + 4)/4 = 2.5$, variance = $((1 - 2.5)^2 + (2 - 2.5)^2 + (3 - 2.5)^2 + (4 - 2.5)^2)/4 = 1.25$.
 - For `h_final`, $\mu_{\text{final}} = (0.5 + 0.1 + 0.9 + 0.7)/4 = 0.55$, variance = 0.0875.
 - Then $\text{var_ratio} = 0.0875/1.25 = 0.07$. $\text{contrast_delta} = |1.25 - 0.0875| = 1.1625$.

Stage 2: Variance	Value	Explanation
Mid-layer variance (<code>var_mid</code>)	1.25	Var of [1,2,3,4]
Final-layer variance (<code>var_final</code>)	0.0875	Var of [0.5,0.1,0.9,0.7]
Variance ratio (<code>var_final/var_mid</code>)	0.07	$0.0875/1.25$
Contrast delta $ \Delta $	1.1625	$ 1.25 - 0.0875 $.

- **Purpose:** A very low `var_ratio` (e.g. <0.0005) indicates the model’s final layer collapsed, which is empirically a strong signal of a forced (possibly malicious) output. We record it to later flag “collapse” cases. In our example, $\text{var_ratio} \approx 0.07 > \text{threshold}$, so no collapse; if it were <0.0005 , code would label a collapse.

Stage 3: Embedding Extraction and Normalization

- **Input:** The final-layer hidden state matrix `h_final` from Stage 1.
- **Operation:** Collapse the $(1 \times N \times d)$ hidden state to a single vector and normalize:
- **Mean over tokens:** Compute `embedding_tensor = mean(h_final, dim=1)`. With shape $(1, d)$, flattening yields a d -dimensional vector of features.
- **Normalize to unit norm:** Let $\mathbf{v}$ be the flattened vector; then

$$\text{embedding} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} \quad (\|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2}).$$

If $\|\mathbf{v}\|$ is zero, leave it as zeros.

- **Output:** A 1-D normalized embedding vector.
- **Example:** From our toy `h_final` above, the token-means are $[0.5, 0.1] \rightarrow 0.3$ and $[0.9, 0.7] \rightarrow 0.8$. So the unnormalized vector is $\mathbf{v} = [0.3, 0.8]$. Its norm is $\|\mathbf{v}\| = \sqrt{0.3^2 + 0.8^2} = 0.854$. The normalized embedding is $\text{embedding} = [0.3/0.854, 0.8/0.854] \approx [0.351, 0.936]$.

Stage 3: Embedding	Values	Explanation
Token-averaged vector	[0.3, 0.8]	Mean of each feature across tokens
Norm of vector	0.854	$\sqrt{0.3^2 + 0.8^2}$
Normalized embedding	[0.351, 0.936]	$\mathbf{v}/\ \mathbf{v}\ $.

- **Purpose:** This “prompt embedding” represents the semantic content of the input in a fixed-dimensional space. All subsequent steps (distance, cosine similarity) use this normalized vector.

Stage 4: Anchor Computation (Benign & Malice Vectors)

- **Input:** Two global buffers: `trusted_manifold` (list of previous *benign* embeddings) and `blocked_embeddings` (list of *malicious/outlier* embeddings). Initially they may be empty.
- **Operation:** Update two “anchor” vectors:
- **research_anchor:** If `trusted_manifold` is nonempty, set it to the normalized mean of all trusted embeddings. In code:

$$\text{research_anchor} = \frac{\sum_{w \in \text{trusted_manifold}} w}{\|\sum w\|}.$$
- **exploit_anchor:** If `blocked_embeddings` has ≥ 5 items, set it to the normalized mean of blocked embeddings. Otherwise (if no blocked set yet) set `exploit_anchor = -research_anchor` (the opposite direction).
- **Output:** Two fixed-length anchor vectors.
- **Example:** Suppose after a few benign prompts we have `trusted_manifold` = {[0.5, 0.5], [0.6, 0.4]}. Then `mean` = [0.55, 0.45], `norm` ≈ 0.714 , and `research_anchor` = [0.55, 0.45]/0.714 $\approx$ [0.771, 0.636]. If `blocked_embeddings` is empty (or < 5), we use `exploit_anchor` = −[0.771, 0.636] $\approx$ [−0.771, −0.636].

Stage 4: Anchors	Values	Explanation
Trusted embeddings	[0.5,0.5], [0.6,0.4]	Example benign embeddings.
Mean vector	[0.55, 0.45]	Average of trusted embeddings.
research_anchor (normed)	[0.771, 0.636]	Normalized mean of benign embeddings.
exploit_anchor (initial)	[-0.771, -0.636]	Negative of research_anchor (since no blocked yet).

- **Purpose:** The **research_anchor** represents the “center” of benign behavior, while **exploit_anchor** represents a malicious direction. By default, an adversary is assumed opposite the benign anchor until enough malicious examples are seen. These anchors let us compute how “aligned” a new prompt is with benign vs. malicious directions.

Stage 5: Distance to Benign Manifold (Nearest Neighbor)

- **Input:** The current prompt embedding (from Stage 3) and the set `trusted_manifold` of prior benign embeddings. Also the `contrast_delta` from Stage 2.
- **Operation:**
- **Initial training period:** If `index < MANIFOLD_FROZEN_AT` (25 in code), simply add the new embedding to `trusted_manifold` (bootstrapping phase) and skip this stage; set `distance=0` and `is_outlier=False`.
- **After warming:** Use scikit-learn's `NearestNeighbors(n_neighbors=1, metric="cosine")` on `trusted_manifold` to find the **cosine distance** to the nearest benign embedding. Cosine distance is defined as $D = 1 - \cos(\theta)$ where $\cos(\theta)$ is the cosine similarity. A distance of 0 means identical direction. In code:

```
nn = NearestNeighbors(n_neighbors=1, metric="cosine").fit(X_safe)
dist_to_safe, _ = nn.kneighbors([embedding])
```

- **Outlier check:** Compare `dist_to_safe` to a threshold. If `contrast_delta < TURB_THRESHOLD` (11000), use threshold 0.026; else use 0.055. Set `is_outlier = (dist_to_safe > threshold)`.
- **Output:** The scalar `dist_to_safe` and boolean `is_outlier`.
- **Example:** Continuing our example, suppose after warmup `trusted_manifold = \{\, [0.3,0.95], [0.1,0.99]\}` (already normalized). Our new embedding was `[0.351, 0.936]` (normalized). Cosine similarity to the nearest neighbor:
 - Compute $\cos(\text{emb}, [0.3, 0.95]) \approx 0.9986$. Then $\text{distance} = 1 - 0.9986 = 0.0014$.
 - Threshold = 0.026 (since `contrast_delta=1.1625 < 11000`). Here $0.0014 < 0.026$, so `is_outlier=False`.The pipeline would set `dist_to_safe ≈ 0.0014` and `is_outlier=False`.

Stage 5: Distance	Value	Explanation
Embedding	[0.351, 0.936]	Current prompt embedding (from Stage 3).
Trusted neighbors	[0.300, 0.953], [0.100, 0.995]	Prior benign embeddings (normalized).
Cosine distance to NN	0.0014	$1 - \cos([0.351, 0.936], [0.300, 0.953])$.
is_outlier?	False	$0.0014 < 0.026$ (threshold) $\Rightarrow$ not outlier.

- **Purpose:** This measures how unusual the prompt's embedding is relative to known benign prompts. A large distance (above threshold) flags the embedding as an *outlier* requiring extra caution.

Stage 6: Cosine Similarity & Margin

- **Input:** The normalized embedding, `research_anchor`, and `exploit_anchor` from Stage 4.

- **Operation:** Compute two cosine similarities:

$$\cos_{\text{exploit}} = \cos(\text{embedding}, \text{exploit_anchor}),$$

$$\cos_{\text{research}} = \cos(\text{embedding}, \text{research_anchor}).$$

Since both vectors are normalized, $\cos(\mathbf{a}, \mathbf{b}) = \mathbf{a} \cdot \mathbf{b}$. Then compute the **margin**:

$$\text{margin} = \cos_{\text{exploit}} - \cos_{\text{research}}.$$

- **Output:** Scalar `exploit_sim`, `research_sim`, and `margin`.

- **Formula:** For normalized vectors a and b ,
 $\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$. (See [18] for the formal definition of cosine similarity.)

- **Example:** With `research_anchor = [0.771, 0.636]` and `exploit_anchor = [-0.771, -0.636]`, our embedding `[0.351, 0.936]` yields:

- $\cos_{\text{research}} = [0.351, 0.936] \cdot [0.771, 0.636] \approx 0.865$.
- $\cos_{\text{exploit}} = [0.351, 0.936] \cdot [-0.771, -0.636] \approx -0.865$.
- $\text{Margin} = -0.865 - 0.865 = -1.730$.

Stage 6: Cosine/Margin	Value	Explanation
<code>research_anchor</code>	<code>[0.771, 0.636]</code>	Mean benign direction.
<code>exploit_anchor</code>	<code>[-0.771, -0.636]</code>	Mean “malicious” direction.
<code>cos_exploit</code>	<code>-0.865</code>	Cosine(embedding, exploit_anchor).
<code>cos_research</code>	<code>+0.865</code>	Cosine(embedding, research_anchor).
Margin (exploit–research)	<code>-1.730</code>	<code>−0.865 − 0.865</code> .

- **Purpose:** The margin measures the net alignment of the prompt: *positive margin* suggests the embedding is closer to malicious than benign, negative margin means closer to benign. This margin then drives further penalties (e.g. if margin is high, intent penalty grows).

Stage 7: Intent Penalty Calculation

- **Input:** The margin from Stage 6, and the current threshold `theta_exploit` (θ). Initially $\theta=0$ until enough benign data accumulates. Also the scale `INTENT_SCALE_K` (500.0 in code).
- **Operation:** Compute

$$\text{intent_penalty} = \max(0, (\text{margin} - \theta) \times \text{INTENT_SCALE_K}).$$

In other words, only margins above θ incur a penalty (scaled by a large constant 500).

- **Output:** Scalar `intent_penalty`.

- **Example:** Suppose $\text{margin} = -1.730$ (as above) and $\theta=0.0$. Then $(\text{margin}-\theta) = -1.730$, which is negative, so the max with 0 yields `intent_penalty = 0.0`. If instead margin had been $+0.2$ with $\theta=0$, then $(0.2-0)*500 = 100$, giving a penalty of 100.

Stage 7: Intent Penalty	Value	Explanation
Margin	-1.730	From Stage 6.
θ_{exploit}	0.0 (initial)	No prior benign margins.
$(\text{margin} - \theta)$	-1.730	Negative $\rightarrow$ no penalty.
Intent penalty	0.0	$\max(0, -1.730 \times 500) = 0$.

- **Purpose:** To **penalize** the prompt if it is unusually close to the “malicious” direction. Only positive margins (indicating potential malicious intent) create a cost, ensuring benign-ish prompts (negative margin) are not penalized.

Stage 8: Manifold Penalty Calculation

- **Input:** The distance `dist_to_safe` from Stage 5, `contrast_delta`, and the boolean `is_outlier`. Also constants `PENALTY_MULT` (120.0) and `TURB_THRESHOLD` (11000).

- **Operation:** If `is_outlier == True`, compute:

$$\text{manifold_penalty} = \text{dist_to_safe} \times \text{PENALTY_MULT} \times f(\text{contrast_delta}),$$

where $f(\text{contrast_delta}) = 1.8$ if $\text{contrast_delta} < \text{TURB_THRESHOLD}$, else 1.0. If not outlier, set `manifold_penalty = 0.0`.

- **Output:** Scalar `manifold_penalty`.

- **Example:** In our example, `is_outlier=False`, so `manifold_penalty=0.0`. If hypothetically `dist_to_safe=1.0` (very far) and $\text{contrast_delta} < 11000$, then penalty would be $1.0 \times 120 \times 1.8 = 216$.

Stage 8: Manifold Penalty	Value	Explanation
<code>dist_to_safe</code>	0.0014	From Stage 5 (very small).
<code>is_outlier</code>	False	Not flagged as outlier.

Stage 8: Manifold Penalty	Value	Explanation
manifold_penalty	0.0	=0 since not outlier.

- **Purpose:** To **penalize** prompts that lie outside the “safe zone” of known benign embeddings. The further from the manifold (higher distance), the bigger the penalty (scaled by 120 and an extra factor if `contrast_delta` is low). This encourages the system to block outlier prompts.

Stage 9: Z-Score (Margin Anomaly) and Convergence Watchdog

- **Input:** The current margin, and the deque `intent_benign_buffer` of recent benign margins. Also the `monitor` (SafetyMonitor) and `watchdog` (ConvergenceWatchdog) objects for logging. The constants `INTENT_BUFFER_WARMUP_DYNAMIC = 25`.
- **Operation:** If the benign buffer has at least 25 values, compute the standard statistical **Z-score** of the current margin relative to that buffer:

$$\text{mean} = \mu = \text{avg}(\text{buffer}), \quad \sigma = \text{std}(\text{buffer}), \quad z = \frac{\text{margin} - \mu}{\sigma + 10^{-9}}.$$

(We add 10^{-9} to σ to avoid division by zero.) This is a standard formula: “Z-score” tells how many standard deviations the margin is from the mean ¹.

Meanwhile, the ConvergenceWatchdog checks drift/inflation of the benign margin distribution. Its logic (in `validate_health`) is:

- If not yet “calibrated”, it records the first mean and std as the baseline.
- Otherwise, it compares the new mean and std to the baseline. If $\text{mean} > 1.1 \times \text{baseline}$, it reports “DRIFT”; if $\text{std} > 1.1 \times \text{baseline}$, it reports “INFLATION”. (These details appear as diagnostics but do not alter the score.)

- **Output:** The scalar `z_score`. Additionally, the watchdog may print status (warm-up, drift, etc.).
- **Example:** If `intent_benign_buffer = [0.1, 0.2, 0.0, 0.15, ...]` with mean $\mu = 0.1$ and std $\sigma = 0.05$, and current margin = -1.730, then

$$z = \frac{-1.730 - 0.1}{0.05} = -36.6.$$

(A very negative outlier, though in practice our buffer is likely small at start so z would be computed later.)

Stage 9: Z-score	Value	Explanation
Benign buffer (last margins)	e.g. [0.10,0.20,...]	Recent benign margins.
Mean (μ)	0.10	Example mean.
Std (σ)	0.05	Example standard deviation.
Current margin	-1.730	From Stage 6.

Stage 9: Z-score	Value	Explanation
Z-score	-36.6	$(-1.73 - 0.1)/0.05$.
Watchdog status	Possibly "DRIFT"	If μ or σ exceed 10% of baseline (not shown).

- **Purpose:** The Z-score identifies *statistical outliers* in margin relative to what has been deemed "normal (benign)" so far. A high absolute Z (e.g. >3) would indicate the prompt is very anomalous. The code uses `z_score` later to impose an **Interlock Penalty** if it exceeds `Z_SCORE_THRESHOLD` (2.5).

Stage 10: Interlock Penalty (Z-Score Penalty)

- **Input:** The `z_score` from Stage 9 and the constant `Z_SCORE_THRESHOLD = 2.5`.
- **Operation:** If `z_score > Z_SCORE_THRESHOLD`, compute

$$\text{interlock_penalty} = (z_score - 2.5) \times 8.0 + 5.0.$$

Otherwise set it to 0.

- **Output:** Scalar `interlock_penalty`.
- **Example:** If `z_score = 3.5`, then $\text{penalty} = (3.5 - 2.5) * 8 + 5 = 1 * 8 + 5 = 13.0$. If `z_score = 2.0`, $\text{penalty} = 0$.

Stage 10: Interlock Penalty	Value	Explanation
<code>z_score</code>	e.g. 3.5	Example from Stage 9.
Threshold (<code>Z_SCORE_THRESHOLD</code>)	2.5	Configured in code.
Penalty	13.0	$(3.5 - 2.5) * 8 + 5$.

- **Purpose:** This strongly penalizes a prompt whose margin is a statistical outlier (i.e. beyond 2.5σ from the mean). It's a kind of safety interlock: the more extreme the outlier, the higher this penalty.

Stage 11: Final Signal and Trust Score

- **Input:** The calculated quantities: `contrast_delta` (from Stage 2), and penalties from Stages 7–10. Constants: none extra.
- **Operation:** First compute a "base signal" from `contrast_delta`:

$$\text{base_signal} = \frac{12.0}{(\text{contrast_delta} / 1000 + 1.0)} + 1.2.$$

Then combine with penalties:

$$\text{final_signal} = \text{base_signal} - (\text{intent_penalty} + \text{manifold_penalty} + \text{interlock_penalty}).$$

Finally compute the **trust score** via a logistic (sigmoid) function:

$$\text{trust_score} = \frac{1}{1 + e^{-\text{final_signal}}}$$

The logistic function $1/(1 + e^{-x})$ maps real signals into (0,1) ². (It's also called the sigmoid.)

- **Output:** A scalar `trust_score` between 0 and 1.

- **Example:** In our running example, say `contrast_delta = 1.1625`. Then `contrast_delta/1000 = 0.0011625`. So

$\text{base_signal} = 12/(0.0011625) + 1.2 \approx 11.986 + 1.2 = 13.186$. Our penalties so far were all zero (`intent_penalty=0`, `manifold_penalty=0`, `interlock_penalty=0`), so `final_signal = 13.186`. The trust score is then

$$\text{trust_score} = 1/(1 + e^{-13.186}) \approx 0.999998.$$

Nearly 1 (almost certainly “safe”).

Stage 11: Trust Score	Value	Explanation
<code>contrast_delta/1000</code>	0.0011625	From Stage 2.
<code>base_signal</code>	13.186	$12/(0.0011625 + 1) + 1.2$.
Sum of penalties	0.0	All zero in this example.
<code>final_signal</code>	13.186	<code>base_signal</code> minus penalties.
<code>trust_score</code> (sigmoid)	0.999998	$1/(1 + e^{-13.186})$.

- **Purpose:** This stage consolidates all previous signals into a single probability-like score. A high base signal (from a non-collapsed variance) tends to give a high trust score, but large penalties will reduce it. The logistic ensures the score remains in [0,1] and moves smoothly. For reference, the logistic function formula and interpretation are standard ².

Stage 12: Purity Gate and Benign Buffer Update

- **Input:** The `trust_score`, `is_outlier` flag, and `z_score`. Also the benign buffer `intent_benign_buffer` (of past margins).

- **Operation:** The code enforces a *Purity Gate*: after the initial warmup, any prompt with `z_score > 1.0` is considered too “edgy” to use as a benign example (even if allowed). Concretely:

- Let `is_pure = True`. If `len(intent_benign_buffer) >= 25` (warmup done) and `z_score > 1.0`, then set `is_pure=False` and do **not** add this margin to the buffer.

- Otherwise, if `trust_score > 0.80` and `is_outlier==False` and `is_pure==True`, **append** the current margin to `intent_benign_buffer`.

This keeps the benign buffer filled only with high-confidence, “pure” prompts.

- **Output:** Possibly appending the margin to the benign buffer.

- **Example:** In our case, `trust_score=0.999998 > 0.8`, `is_outlier=False`, and suppose `z_score=-36.6` (so `<1`). Then all conditions pass, so we do `intent_benign_buffer.append(-1.730)`.

Stage 12: Purity Gate	Condition	Action
<code>trust_score (>0.8)</code>	True ($0.9999 > 0.8$)	OK
<code>is_outlier (False)</code>	True	OK
<code>z_score (>1.0?)</code>	No ($-36.6 \leq 1.0$)	OK
Result	All true	Append margin to benign buffer

- **Purpose:** This step updates our benign statistics. By only adding “pure” margins (high trust, non-outlier), the system’s model of normal margins stays robust. This buffer is later used in Stage 9 (Z-score) and in updating θ_{exploit} .

Stage 13: Exploit Threshold Update

- **Input:** The updated `intent_benign_buffer` of benign margins. Constant `INTENT_PERCENTILE = 97.5`.
- **Operation:** If the benign buffer has ≥ 25 entries, update the global threshold `theta_exploit` to the 97.5-th percentile of the buffer. In code:
 $\theta_{\text{exploit}} = \max(\text{percentile}(\text{intent_benign_buffer}, 97.5), 0)$.
- **Output:** New scalar `theta_exploit`.
- **Example:** If benign margins so far are `[-0.5, 0.0, 0.1, 0.2, ...]` (25 numbers), the 97.5th percentile might be around `+0.48` (for example). Then future margins must exceed `~0.48` before they incur intent penalty.
- **Purpose:** This dynamically adjusts how “large” a margin must be before it’s considered suspicious. By setting it near the upper end of benign margins, only exceptionally large positive margins (malicious-like) get penalized.

Stage 14: Blocked Embeddings Update

- **Input:** The `is_outlier` flag and `trust_score`. The global deque `blocked_embeddings`.
- **Operation:** If `is_outlier==True` and `trust_score < 0.5`, then append the current embedding to `blocked_embeddings`. This records the embedding of strongly malicious (blocked) prompts.
- **Output:** Possibly update `blocked_embeddings`.

- **Example:** In our example nothing happens (not outlier, trust ~1). If instead a prompt was flagged (dist far) and trust low, it would join the malicious set.
- **Purpose:** This builds a set of examples thought to be attacks. Once it has ≥ 5 entries, future prompts will compute `exploit_anchor` as their average (Stage 4).

Stage 15: Output

Finally, the function prints diagnostics and returns the single value `trust_score`. A value >0.5 means "ALLOW", else "BLOCK" (in code's logic). This is the end of the pipeline for one prompt.

Auxiliary – Convergence Watchdog: Separately, the `ConvergenceWatchdog` (used in Stage 9) ensures the benign margin distribution remains stable. It compares the current mean (μ) and std (σ) of the benign buffer to the initial baseline. If either drifts by $>10\%$, it prints a warning. This does **not** change the trust score but is reported for monitoring model "health".

Example Walkthrough (Single Prompt)

Prompt: "Explain DNS hijacking." (benign query)

1. **Tokenize & Model:** The prompt is tokenized and run through the model, yielding hidden states `h_mid` and `h_final`.
2. **Variance:** Suppose these have variances `var_mid=1.25`, `var_final=0.0875` (as above), giving `var_ratio=0.07`, `contrast_delta=1.1625`. No collapse flagged ($0.07 > 0.0005$).
3. **Embedding:** Compute token-mean vector `[0.3, 0.8]`, normalize to `[0.351, 0.936]`.
4. **Anchor:** If this is early, anchors are uninitialized; assume this is later, we compute `research_anchor` $\approx [0.771, 0.636]$ and `exploit_anchor` $\approx [-0.771, -0.636]$.
5. **Distance:** Nearest safe distance ≈ 0.0014 , below threshold $\Rightarrow$ `is_outlier=False`.
6. **Cosines:** `cos_research` ≈ 0.865 , `cos_exploit` ≈ -0.865 , `margin` ≈ -1.730 .
7. **Intent Penalty:** Margin negative $\Rightarrow$ `intent_penalty=0.0`.
8. **Manifold Penalty:** Not outlier $\Rightarrow$ `manifold_penalty=0.0`.
9. **Z-score:** If buffer exists, likely a large negative z. For simplicity assume no buffer (warmup), so `z_score=0.0`. (ConvergenceWatchdog reports "WARMING_UP".)
10. **Interlock Penalty:** `z_score < 2.5` $\Rightarrow$ `interlock_penalty=0.0`.
11. **Final Score:** `base_signal` ≈ 13.186 , all penalties 0 $\Rightarrow$ `final_signal=13.186`, `trust_score` ≈ 0.999998 .
12. **Purity:** High trust, non-outlier, $z < 1 \Rightarrow$ append margin -1.730 to benign buffer.
13. **Theta Update:** If buffer reached 25 items, update θ . (Otherwise skip.)
14. **Block Update:** Not applicable.
15. **Result:** The prompt is classified **ALLOW** with score ≈ 1.0 .

Each transformation above is supported by standard statistical or geometric formulas (variance, mean, normalization, cosine similarity, z-score, logistic) ¹ ³ ². Edge cases to note: before warmup, many stages are inactive (no Z-score or theta update). If embeddings are identical, distances are 0. If `blocked_embeddings` has <5 vectors, the exploit anchor is simply the negated benign anchor. We

assume typical behavior of `torch.var`, `np.linalg.norm`, etc., and that missing-value or underflow cases (like zero norm) are handled by added constants (as in code).

Visual Summary: The mermaid diagram above outlines the flow from input prompt to final trust score. The key insights are that the system uses *geometric* measures (cosine distances) and *statistical* measures (Z-scores, percentiles) to flag anomalous prompts. It maintains dynamic buffers of “benign” margins and computes a moving threshold (θ_{exploit}) to scale penalties. In effect, a prompt is blocked if it lies far from the benign cluster or strongly resembles malicious examples.

Assumptions: We assumed 2D embeddings for illustration; in reality `d` could be hundreds. We treated threshold constants (0.026, 0.055, $Z_{\text{thresh}}=2.5$) as given. We assumed tokens have been preprocessed normally. If any buffer is empty, certain operations skip (e.g. Z-score=0).

Sources: All transformations (variance, mean, normalization, cosine similarity, percentile, Z-score, logistic function) follow standard definitions ¹ ³ ² ⁴. The scikit-learn `NearestNeighbors` with `metric="cosine"` computes $1 - \cos \theta$ distance ⁵. These citations document the formulas used.

¹ Normal distribution - Wikipedia

https://en.wikipedia.org/wiki/Normal_distribution

² Logistic function - Wikipedia

https://en.wikipedia.org/wiki/Logistic_function

³ Cosine similarity - Wikipedia

https://en.wikipedia.org/wiki/Cosine_similarity

⁴ ⁵ 1.6. Nearest Neighbors — scikit-learn 1.8.0 documentation

<https://scikit-learn.org/stable/modules/neighbors.html>